Lctr 6. General-Purpose Input and Output (GPIO)

Jianhua Liu

Spring 2026

==Topics==

Core concepts and applications:

- GPIO hardware composition.
- GPIO registers.
- GPIO programming.
  - Data structure of GPIO registers.
  - Pointer to data structure of GPIO registers.
  - GPIO functions.

Demo project:

- Controlling an LED (GPIO output) via a pushbutton (GPIO input).
- Controlling an LED via UART.

Additional reading:

- GPIO modes.
  - GPIO output modes—push-pull or open-drain.
  - GPIO input modes—pull-up, pull-down, or no-pull.

6.1 Introduction

A GPIO (general-purpose input and output) pin of an ARM Cortex-M processor can perform the following functions:

- **Digital input** that detects whether an external voltage signal is higher or lower than a predetermined threshold—the corresponding reading will be logic 1 and logic 0, respectively.
- **Digital output** that controls the output voltage level on the pin.
- **Analog functions** that perform digital-to-analog or analog-to-digital conversions.
- **Other complex functions** such as timer-based input capture, external interrupt, as well as hardware interface of UART and USB communications.

In this lecture we only consider the first two. Some of the other topics will be addressed later.

6.2 GPIO hardware composition

6.2.1 GPIO ports

6.2.2 A brief overview of the electronic components

We use the following electronic components in the discussion below.

- **Resistor**, a device in which the current can flow in both directions.
- **Diode**, a device that only allows the current to flow from the anode to the cathode. An LED can be seen as a special type of diode.
- **Transistor**, a device that can be used to control current flow. Here, we mainly use it as a switch: closed (on) or open (off). There are two types of transistors we often see. Here we only discuss one:
  - MOSFET (metal-oxide-semiconductor field-effect transistor). It has three terminals: the gate (G), the source (S), and the drain (D). We can control the current flowing from the drain to the source by changing the value of the **gate voltage**.

6.2.3 Main components for a GPIO pin

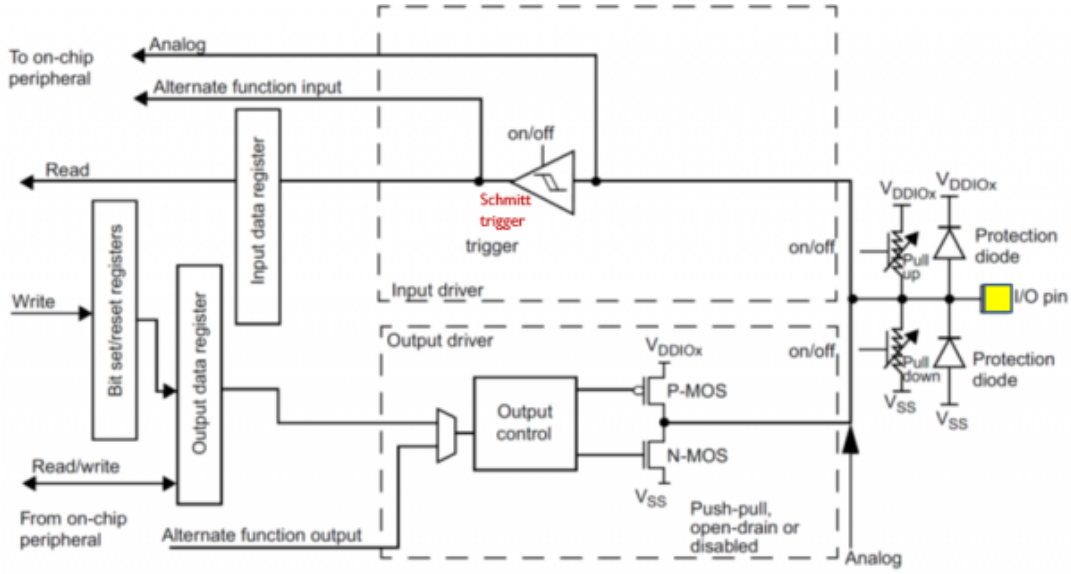


Figure 1: Schematic of the circuit for a GPIO pin.

Figure 1 shows the schematic of the hardware needed to support a GPIO pin, which includes several main components.

6.2.4 Non-controllable vs controllable components

The above components are divided into two classes: Non-controllable components and controllable ones.

ESD diodes and the Schmitt trigger are non-controllable components. Their functionalities are defined by hardware design and cannot be controlled by the MCU itself.

Pull-up/down resistors and the final state of the output driver are controllable ones, and they are controlled by the MCU for intended functionalities. Let's take a look at how they are used below.

6.2.5 Pull-up, pull-down, or no connection for input

Assume we set up the GPIO pin shown in Figure 1 to **digital input**. Then, the (input) **voltage** applied to the pin can have one of the following three states:

- **High (value)**, which is read as a solid **logic 1** by the Schmitt trigger, which will be simplified as 1 in the sequel.
- **Low (value)**, which is read as a solid **logic 0** by the Schmitt trigger, which will be simplified as 0 in the sequel.
- **High impedance**, also called floating or HiZ (Z is the common notation for impedance), which can be read by the MCU as 0 or 1 **randomly**. This can create a lot of troubles for the processing of the input.

To handle the issue caused by the high impedance, we often need to *force* the input be read as 0 or 1 **consistently**, depending on the design preference.

- If we want a HiZ input to be read as *high*, we need to have a **pull-up** resistor, effectively *connecting* the input to the power source.
- If we want a HiZ input to be read as *low*, we need to have a **pull-down** resistor, effectively *connecting* the input to the ground.

Note that in addition to the above **pull-up** and **pull-down** setups, there is one more setup: **no connection**. The control of these **three situations** requires 2 bit in the **Pull-up/down register** (PUPDR), as we will see later.

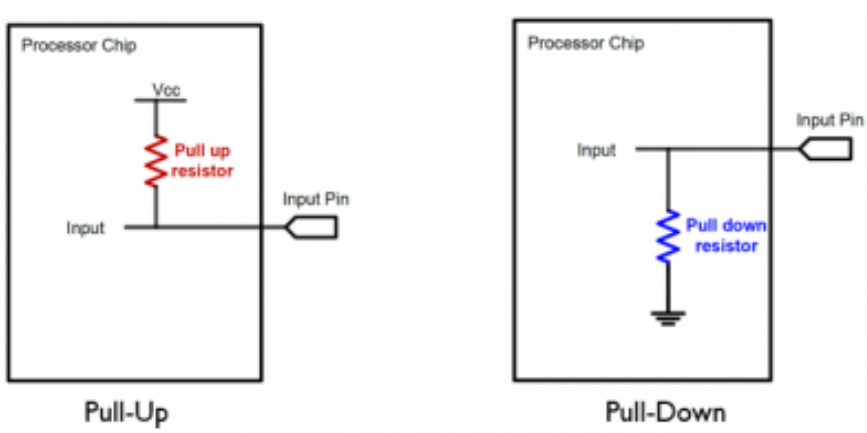


Figure 2: Illustration of MCU internal pull-up and pull-down circuit.

6.2.6 Push-pull or open-drain for output

There are **two cases for the GPIO output**: **push-pull** and **open-drain**.

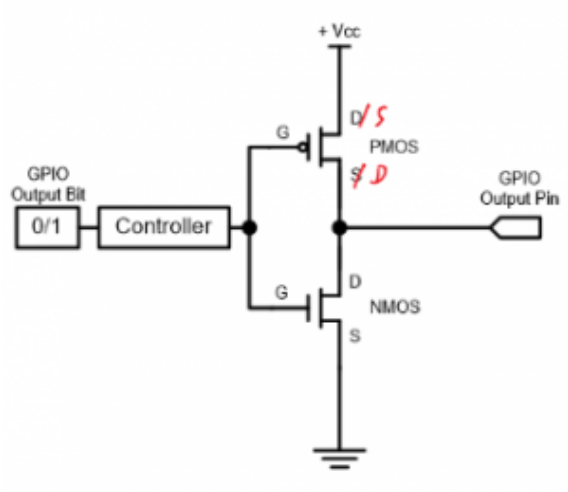
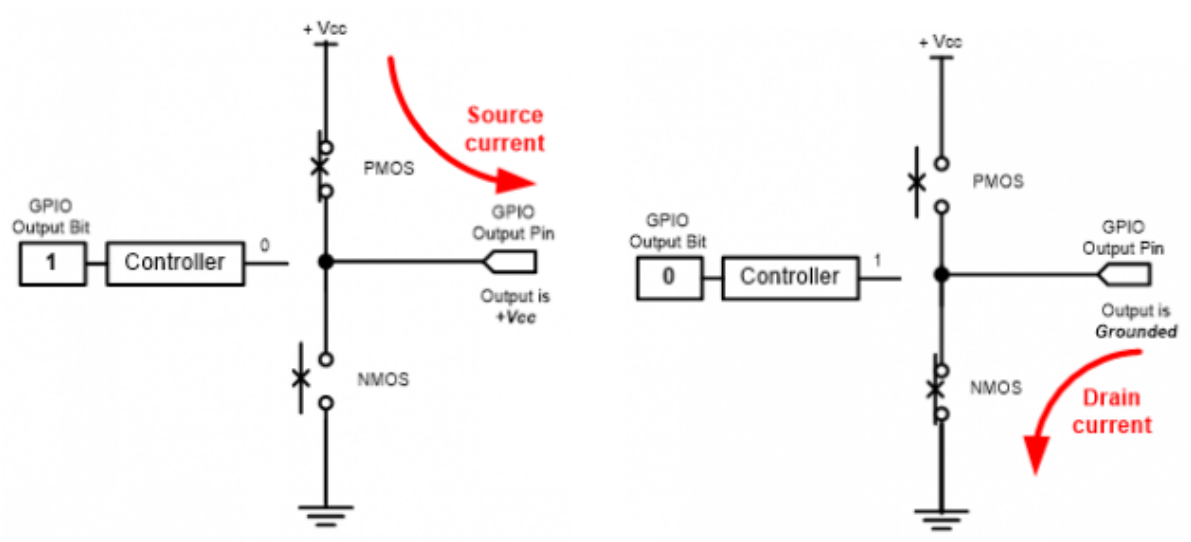


Figure 3: Schematic for a push-pull output circuit.



The open-drain circuit is the same as the push-pull circuit except the top transistor is removed.

When we want to output logic 1, the controller turns off the bottom transistor. This way, the **output is floating or HiZ**, as shown in Figure 4. The open-drain circuits need a pull-up *resistor* which pulls the output to a steady high.

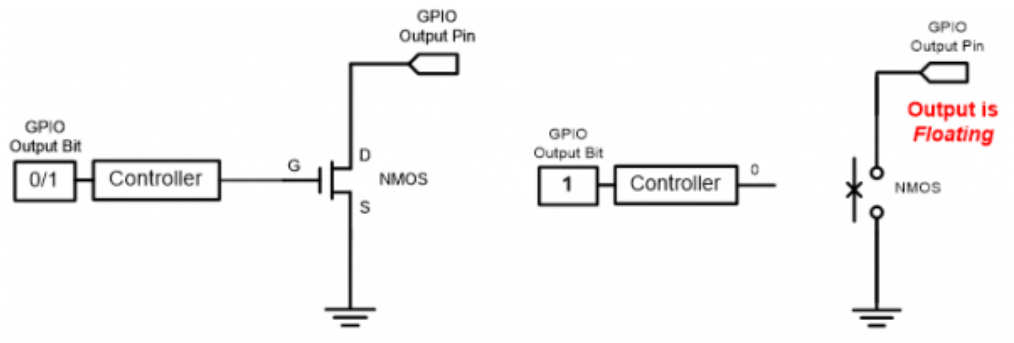


Figure 4: Equivalent circuit for an open-drain circuit set up to be 1 (ON).

When we want to output logic 0, the controller turns on the bottom transistor. This way, it pulls a current from the load, forcing the output to be logic 0, as shown in Figure 5.

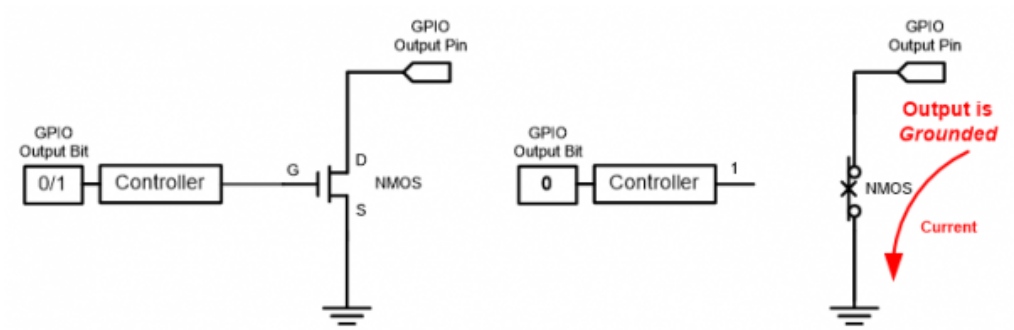


Figure 5: Current flow for an open-drain circuit set up to be 0 (OFF).

Note that we only need one bit in the **Output type register** (OTYPER) to choose the type of output, as we will see later.

6.2.7 Comparisons and contrast of similar circuits

Now, let us compare some similar circuits or totally different circuits with similar terminologies to remove potential confusions.

6.2.7.1 Push-pull vs open-drain

1. The circuit of open-drain is the same as that of the push-pull except it lacks the top switch (PMOS), the (current) pushing part.
2. Open-drain can only pull current from the output pin, whereas push-pull can also push a current to the load from the output pin in addition to pull current from the output pin.
3. Open-drain can be used to change the level of the logic.
4. Open-drain can be used to drive the load with higher voltages.
5. **Open-drain can be used to do wired-AND logic operations**. In this case, several open-drain output units can be connected together directly. If one of the output is low, the entire output is dragged down to be low. Only if all the outputs are high, which means no one can draw a current, the output can be high due to the pull-up resistor (or a similar device). This works like the AND logic gate.

6.2.7.2 Push-pull vs pull-up/down

1. **Push-pull is for output**. It provides the needed current to the load via connecting the load to the voltage source or ground via a switch (the MOSFET). It can handle a current up to 30 mA depending on the device.
2. **Pull-up and pull-down are for input**. They are used to remove the uncertainty when the input is floating—high impedance.
   - If we want a floating input to be seen as solid 0, we need to use pull-down.
   - If we want a floating input to be seen as solid 1, we need to use pull-up.
   - When there is no need to remove this type of uncertainty, we do not need to use either. The current of a pull-up/down connection is only in the sub-mA level.

6.3 Demo project

This project, `dc2s_gpio_key_n_led__uart`, uses two GPIO pins on G431n32 and similar MPBs:

- PA0 (Pin 0 of GPIO Port A), labeled as Fnc6 (function key No. 6 on the game controller) for GPIO input.
  - PA0 will be connected to 3.3 V when it is pressed, which reads as 1; as a result, it should be configured as **pull-down** to read as 0 when the key is not pressed.
- PB8 (Pin 8 of GPIO Port B), labeled as LED4 (on the game controller) for GPIO output.
  - It is configured as **push-pull** since we need to push a current to turn on the LED.

This project has two main functionalities.

- **Functionality controlled by a pushbutton**, also called key. We use Fnc6 to control the blinking of LED4. When the key is pressed, LED4 blinks twice.
- **Functionality controlled by UART**. We use UART input to turn on or off the LED. We are prompted to input 0 or 1 in the terminal:
  - 0 will turn off LED4.
  - 1 will turn on LED4.

6.4 GPIO Registers, data structure, and pointers

We have seen the schematic of the circuit for a GPIO pin, which is duplicated here in Figure 6. Shown in the left part of the figure are three GPIO registers, from right to left:

- **Input data register** (IDR), used to read (1 or 0) from the corresponding GPIO pins when the GPIO works in the GPIO input mode.
- **Output data registers** (ODR), used to write 1 or 0 to the corresponding GPIO pins when the GPIO works in the GPIO output mode.
- **Bit set/reset registers** (BSRR), used to write 1 to the appropriate bits in the register to change the bits in ODR in a faster way. Details will be discussed later.

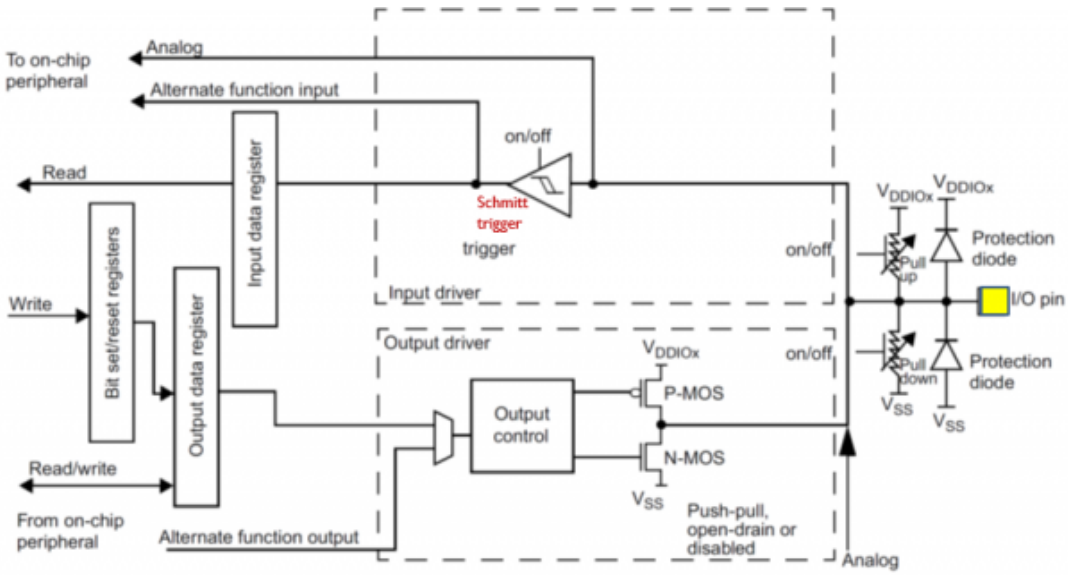


Figure 6: Schematic of the circuit for a GPIO pin.

There are other GPIO registers, as will be seen later. All the functionality of the GPIO ports and pins are controlled by GPIO registers.

6.4.1 Memory-mapped peripheral control

ARM Cortex-M processors use memory-mapped GPIO registers, where each GPIO port is assigned a number of registers in the data memory space of the MCU in a systematic way.

This idea is extended to all the other hardware peripherals.

6.4.2 A list of GPIO registers

For a family of MCUs from a certain manufacture, they define the same GPIO registers for each GPIO port. Below is a list of all the GPIOA (GPIO port A) registers and their addresses for a Cortex-M4-based MCU from STMicroelectronics. (See `stm32g431xx.h`.)

Address	Register	Description
`0x4800_0000:`	`MODER`	Mode register
`0x4800_0004:`	`OTYPER`	Output type register
`0x4800_0008:`	`OSPEEDR`	Output speed register
`0x4800_000C:`	`PUPDR`	Pull-up/down register
`0x4800_0010:`	`IDR`	Input data register
`0x4800_0014:`	`ODR`	Output data register
`0x4800_0018:`	`BSRR`	Bit set/reset register
`0x4800_001C:`	`LCKR`	Config lock register
`0x4800_0020:`	`AFR[2]`	Alternate function registers
`0x4800_0028:`	`BRR`	Bit reset register

Before moving on, let's make one more point. When used as digital output or input, **each GPIO pin can be controlled independently**.

6.4.3 Data structure for registers of a GPIO port

The registers for a GPIO port is defined in a register bank using the following struct:

```
typedef struct {
    __IO uint32_t MODER;        // Address offset: 0x00
    __IO uint32_t OTYPER;       // Address offset: 0x04
    __IO uint32_t OSPEEDR;      // Address offset: 0x08
    __IO uint32_t PUPDR;        // Address offset: 0x0C
    __IO uint32_t IDR;          // Address offset: 0x10
    __IO uint32_t ODR;          // Address offset: 0x14
    __IO uint32_t BSRR;         // Address offset: 0x18
    __IO uint32_t LCKR;         // Address offset: 0x1C
    __IO uint32_t AFR[2];       // Address offset: 0x20-0x24
    __IO uint32_t BRR;          // Address offset: 0x28
} GPIO_TypeDef;
```

Note that each GPIO port controls up to 16 pins, and hence the IDR and ODR are actually 16-bit wide. The above data structure can be alternately defined as shown below to reflect the details, which is usually not needed as it is not as compact and clean as the former.

```
typedef struct {
    __IO uint32_t MODER;        // Address offset: 0x00
    __IO uint32_t OTYPER;       // Address offset: 0x04
    __IO uint32_t OSPEEDR;      // Address offset: 0x08
    __IO uint32_t PUPDR;        // Address offset: 0x0C
    __IO uint16_t IDR;          // Address offset: 0x10
    uint16_t temp;              // Address offset: 0x12
    __IO uint32_t ODR;          // Address offset: 0x14
    ...
} GPIO_TypeDef;
```

6.4.4 Definition of pointers to GPIO register structs

The following code snippets from `stm32g431xx.h` demonstrate how the pointers to GPIO register structs are defined in C. Note that all the addresses are determined by hardware design specifications, some from Arm and some from chip manufacturers.

Memory map and base address for the peripheral:

```
/** @addtogroup Peripheral_memory_map
  * @{
  */
#define FLASH_BASE  ((uint32_t)0x08000000) // FLASH base address
#define SRAM1_BASE  ((uint32_t)0x20000000) // SRAM1 base address
#define PERIPH_BASE ((uint32_t)0x40000000) // Peripheral address
...
```

Memory banks and base addresses for different peripheral buses:

```
/*!< Peripheral memory map */
#define APB1PERIPH_BASE        PERIPH_BASE
#define APB2PERIPH_BASE       (PERIPH_BASE + 0x00010000UL)
#define AHB1PERIPH_BASE       (PERIPH_BASE + 0x00020000UL)
#define AHB2PERIPH_BASE       (PERIPH_BASE + 0x08000000UL)
```

Note that there are different peripheral buses for peripherals of different speeds:

- APB (advanced peripheral bus) is used for low speed peripherals, such as UART.
- AHB (advanced high-performance bus) is used for high speed peripherals, such as GPIO.

Base addresses for GPIO ports on AHB 2:

```
#define GPIOA_BASE            (AHB2PERIPH_BASE + 0x0000UL)
#define GPIOB_BASE            (AHB2PERIPH_BASE + 0x0400UL)
#define GPIOC_BASE            (AHB2PERIPH_BASE + 0x0800UL)
...
```

Pointers to structs of GPIO registers:

```
#define GPIOA               ((GPIO_TypeDef *) GPIOA_BASE)
#define GPIOB               ((GPIO_TypeDef *) GPIOB_BASE)
#define GPIOC               ((GPIO_TypeDef *) GPIOC_BASE)
...
```

6.5 GPIO programming

When we use CubeMX to set up the above pins, the *initialization* code for setup will be generated for us according to the GPIO setups. If we need to do the initialization manually, we need to use the following two functions, mainly the first one:

```
/* Initialization and de-initialization functions
   ******************/
void HAL_GPIO_Init(GPIO_TypeDef  *GPIOx, GPIO_InitTypeDef *
   GPIO_Init);
void HAL_GPIO_DeInit(GPIO_TypeDef  *GPIOx, uint32_t GPIO_Pin);
```

These functions are defined in the `stm32g4xx_hal_gpio.c` and `stm32g4xx_hal_gpio.h` files.

The above C source and header files also define the following functions which can be used in the *application* step.

```
/* IO operation functions
   ****************************************/
GPIO_PinState HAL_GPIO_ReadPin(GPIO_TypeDef *GPIOx, uint16_t
   GPIO_Pin);
void HAL_GPIO_WritePin(GPIO_TypeDef *GPIOx, uint16_t GPIO_Pin,
    GPIO_PinState PinState);
void HAL_GPIO_TogglePin(GPIO_TypeDef *GPIOx, uint16_t GPIO_Pin
   );
```

6.6 Additional reading

6.6.1 Bit definition of the GPIO Input Data register

To illustrate the bit definition of 1-bit-per-pin GPIO registers, we use the IDR (the GPIO Input Data register) shown in Figure 7 as an example.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ID15	ID14	ID13	ID12	ID11	ID10	ID9	ID8	ID7	ID6	ID5	ID4	ID3	ID2	ID1	ID0
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

Figure 7: Bit assignment of the GPIO IDR.

6.6.2 Bit definition of the GPIO Mode register

To illustrate the bit definition of 2-bit-per-pin GPIO registers, we use the MODER (the GPIO Mode register) shown in Figure 8 as an example.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
MODE15[1:0]		MODE14[1:0]		MODE13[1:0]		MODE12[1:0]		MODE11[1:0]		MODE10[1:0]		MODE9[1:0]		MODE8[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
MODE7[1:0]		MODE6[1:0]		MODE5[1:0]		MODE4[1:0]		MODE3[1:0]		MODE2[1:0]		MODE1[1:0]		MODE0[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:0 **MODE[15:0][1:0]:** Port x configuration I/O pin y (y = 15 to 0)
These bits are written by software to configure the I/O mode.
00: Input mode
01: General purpose output mode
10: Alternate function mode
11: Analog mode (reset state)

Figure 8: Bit assignment of the GPIO Mode register.

6.6.3 Other common registers

We have discussed that there are only two output types: push-pull and open-drain. Hence, we can use a single bit to control the output type for each pin in the OTYPER (the output type register). So the structure of OTYPER is similar to that of the IDR.

We have also discussed that there are 3 types of pull up/down connections: pull-up, pull-down, or no-pull. As such, we need to have two bits for controlling the pull mode of each pin. So, the structure of PUPDR (Pull-up/down register) is similar to that of the MODER.

6.6.4 Clock-control register

To save power, the circuits for GPIO ports are turned off by default. If we want to use a specific GPIO port, we need to enable its clock to power the corresponding circuit. This can be done by the code generated by CubeMX.